



EAST AFRICA UNIVERSITY
FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY

**DESIGN AND IMPLEMENTATION OF A REAL-TIME
PROJECT AND TASK MANAGEMENT SYSTEM**

BY
MAXAMUUD CABDIQAADIR MAXAMED
3362

**A RESEARCH PROJECT SUBMITTED TO THE FACULTY OF
COMPUTING AND
INFORMATION TECHNOLOGY IN PARTIAL FULFILLMENT OF
THE REQUIREMENTS FOR
THE AWARD OF BACHELOR OF SCIENCE (BSc) DEGREE IN
INFORMATION
TECHNOLOGY OF EAST AFRICA UNIVERSITY**

SEPTEMBER, 2026

DECLARATION

I/We declare that this research project is my/our original work and has not been presented, in part or in whole, to any other academic institution for the award of a degree or any other academic credential.

Name: MAXAMUUD CABDIQAADIR MAXAMED

Signature: _____

Date: _____

APPROVAL

This research project titled “Design and Implementation of a Real-Time Project and Task Management System” has been submitted for examination with my/our approval as the University Supervisor.

Supervisor's Name: Cabdirisaaq Jirde

Signature: _____

Date: _____

DEDICATION

I dedicate this research project to my beloved family, whose endless support, patience and encouragement have been the foundation of my academic journey, and to everyone who believed in my potential to complete this work.

ACKNOWLEDGEMENT

First and foremost, I thank Almighty Allah for granting me the strength, knowledge and perseverance to complete this research project.

I would like to express my sincere gratitude to my supervisor, Cabdirisaaq Jirde, for the guidance, patience and constructive feedback provided throughout the development of this study.

I also extend my appreciation to the entire faculty of the Faculty of Computing and Information Technology, East Africa University, for the knowledge imparted to me during my studies.

Finally, I am grateful to my family, friends and colleagues for their moral support and encouragement throughout this journey.

ABSTRACT

Effective coordination of projects and tasks remains a persistent challenge for many organizations that still rely on manual methods such as spreadsheets, paper-based logs, and informal messaging to plan, assign, and track work. These methods are prone to miscommunication, lost updates, missed deadlines, and a lack of visibility into who is responsible for what, which ultimately reduces organizational productivity and accountability. This study set out to design and implement a role-based, web-based Project and Task Management System capable of centralizing project planning, task assignment, progress tracking, deadline management, and reporting in a single platform accessible to Administrators, Project Managers, and Team Members.

The study adopted the Agile-oriented Software Development Life Cycle (incremental development) as its research and system development design, combined with a Clean Architecture approach for the system's technical design. Requirements were gathered through review of existing literature and analysis of common organizational project-management practices, and were translated into functional and non-functional requirements, entity-relationship models, and use-case designs. The resulting system was implemented using ASP.NET Core 9 MVC, Entity Framework Core 9, and Microsoft SQL Server, and provides role-scoped dashboards, project and task CRUD operations, team assignment, comments and file attachments, automated deadline notifications, exportable PDF/Excel reports, analytics, audit logging, and global search.

The system was tested using unit and integration tests to verify that each module performs its intended function and that the modules interact correctly end-to-end. The findings show that a centralized, role-based platform can substantially reduce the coordination overhead associated with manual project tracking, improve the traceability of task ownership, and give managers real-time visibility into project and team performance. The study concludes with

recommendations for further enhancement of the system, including native mobile access, real-time push notifications, and integration with third-party calendar and communication tools.

Keywords: Project Management, Task Management, Role-Based Access Control, ASP.NET Core, Clean Architecture, Software Development Life Cycle.

TABLE OF CONTENTS

DECLARATION.....	i
APPROVAL	ii
DEDICATION.....	iii
ACKNOWLEDGEMENT.....	iv
ABSTRACT.....	v
LIST OF TABLES	vi
LIST OF FIGURES	vii
LIST OF ABBREVIATIONS	viii
CHAPTER ONE: INTRODUCTION	1
1.1 Background of the Study	1
1.2 Statement of the Problem.....	2
1.3 General Objective of the Study.....	3
1.4 Specific Objectives of the Study.....	3
1.5 Research Questions	3
1.6 Conceptual Framework.....	4
1.7 Scope of the Study	5
1.8 Significance of the Study	5
1.9 Expected Limitations of the Study.....	6
CHAPTER TWO: LITERATURE REVIEW.....	7
CHAPTER THREE: RESEARCH METHODOLOGY	7
CHAPTER FOUR: DATA PRESENTATION, ANALYSIS AND DISCUSSION OF FINDINGS	7

CHAPTER FIVE: CONCLUSION AND RECOMMENDATIONS	7
REFERENCES.....	7
APPENDICES	7

LIST OF TABLES

*(To be completed as tables are introduced in Chapters Three and Four – e.g.
Table 3.1: Summary of Functional Requirements.)*

LIST OF FIGURES

(To be completed as figures are introduced in Chapters Three and Four – e.g. Figure 1.1: Conceptual Framework, Figure 3.1: System Architecture Diagram, Figure 3.2: Entity-Relationship Diagram, Figure 3.3: Use Case Diagram.)

LIST OF ABBREVIATIONS

Abbreviation	Meaning
PTMS	Project and Task Management System
SDLC	Software Development Life Cycle
MVC	Model-View-Controller
ORM	Object-Relational Mapping
ERD	Entity-Relationship Diagram
RBAC	Role-Based Access Control
EAU	East Africa University
UI	User Interface
UX	User Experience
CRUD	Create, Read, Update, Delete
PDF	Portable Document Format
SQL	Structured Query Language

CHAPTER ONE: INTRODUCTION

1.1 Background of the Study

Globally, the growth of organizations across sectors such as construction, information technology, non-governmental work, banking and education has been accompanied by an increasing volume and complexity of projects that must be planned, executed and monitored within limited time and resources. Project management, understood as the application of knowledge, skills, tools and techniques to project activities in order to meet defined requirements, has therefore become a central organizational function (Project Management Institute, 2021). As organizations scale, the number of concurrent projects, tasks and contributors grows, and coordinating who is doing what, by when, and to what standard becomes increasingly difficult to manage through informal means.

Historically, and still today in many organizations, project and task coordination is carried out using spreadsheets, email threads, paper-based logs, and verbal or chat-based instructions. While these tools are familiar and require no specialized training, they do not scale well: spreadsheets are edited by one person at a time and quickly become outdated; email and chat messages bury task assignments and updates in unstructured conversation threads; and paper logs cannot be searched, shared in real time, or backed up reliably. As a direct consequence, tasks are duplicated or forgotten, deadlines are missed without early warning, accountability for specific pieces of work becomes unclear, and managers lack a single, trustworthy view of how a project is actually progressing.

The emergence of web-based information systems has offered a practical response to these coordination problems. A web-based Project and Task Management System (PTMS) allows an organization to centralize project data, assign and track tasks against specific individuals, attach relevant files and

comments directly to the work they concern, and receive automated reminders as deadlines approach — all accessible from any device with a browser, without organization-wide software installation. Such systems also make it possible to enforce role-based access control (RBAC), so that Administrators, Project Managers and Team Members each see and can act on only the information relevant to their responsibilities, which improves both security and usability (Kruchten, 2019).

This study is motivated by the continued reliance of many organizations on informal, manual approaches to project and task coordination despite the availability of mature web technologies capable of solving the underlying problem. It responds to that gap by designing and implementing a role-based, web-based Project and Task Management System — built on ASP.NET Core 9 MVC, Entity Framework Core and Microsoft SQL Server, and structured according to Clean Architecture principles — that centralizes project and task administration, team assignment, deadline tracking, reporting and analytics, and audit logging in a single, secure platform.

1.2 Statement of the Problem

Despite growing awareness of formal project management practice, many small and medium-sized organizations continue to coordinate projects and tasks through disconnected, manual tools such as spreadsheets, verbal instructions and generic messaging applications. This creates several persistent problems. First, task ownership is often ambiguous: because assignments are not recorded in a structured, queryable system, it is difficult to determine, at any given moment, who is responsible for a specific task and what its current status is. Second, deadlines are tracked informally, so there is no automated mechanism to warn a Team Member, Project Manager or Administrator that a task is approaching or has passed its due date, which results in avoidable delays. Third, supporting materials such as comments, clarifications and file attachments

relevant to a task are scattered across email, chat applications and personal folders rather than being attached to the task itself, making it difficult to reconstruct the history and context of a piece of work. Fourth, managers lack real-time, consolidated visibility into project and team performance, since producing a status report typically requires manually compiling information from multiple disconnected sources. Finally, without centralized, role-based access control, sensitive project information can be seen or modified by individuals who should not have that level of access, and there is no reliable audit trail of who changed what and when.

These problems collectively reduce organizational productivity, delay project delivery, weaken accountability and increase the risk of miscommunication. It is against this background that this study designs and implements a role-based, web-based Project and Task Management System intended to address these specific shortcomings by centralizing project and task data, automating deadline tracking, attaching comments and files directly to tasks, enforcing role-based access control, providing real-time dashboards and exportable reports, and maintaining a full audit log of system activity.

1.3 General Objective of the Study

The general objective of this study is to design and implement a role-based, web-based Project and Task Management System that centralizes project planning, task assignment, progress tracking, deadline management, reporting and audit logging for Administrators, Project Managers and Team Members.

1.4 Specific Objectives of the Study

1. To analyze the requirements of a role-based project and task management system, including the functional needs of Administrators, Project Managers and Team Members and the data relationships among users, projects, tasks, comments, attachments and notifications.

2. To design and develop a web-based system, structured according to Clean Architecture, that provides project and task management, team assignment, deadline tracking, notifications, comments and file attachments, reporting/analytics, and audit logging.
3. To test and evaluate the developed system through unit and integration testing in order to verify that its modules function correctly individually and in combination.

1.5 Research Questions

- What are the functional and data requirements of a role-based project and task management system for Administrators, Project Managers and Team Members?
- How can a web-based project and task management system be designed and developed, using Clean Architecture, to provide project/task management, team assignment, deadline tracking, notifications, reporting and audit logging?
- To what extent does the developed system function correctly, as verified through unit and integration testing, in supporting project and task coordination?

1.6 Conceptual Framework

The conceptual framework of this study illustrates the relationship between the independent variables (the core features of the Project and Task Management System) and the dependent variable (effective project and task coordination). It shows that role-based access control, centralized task assignment and tracking, automated deadline notifications, and integrated reporting and analytics jointly act on organizational processes to produce improved coordination, accountability and visibility, while intervening variables such as user digital

literacy, internet connectivity and organizational change-readiness may moderate the strength of this relationship.

INDEPENDENT VARIABLES (System Features)		DEPENDENT VARIABLE (Outcome)
• Role-based access control • Project & task assignment • Deadline tracking & notifications • Comments & attachments • Reporting, analytics & audit logging	→	Effective, accountable and transparent project and task coordination within the organization

Figure 1.1: Conceptual Framework of the Study (Source: Researcher, 2026)

1.7 Scope of the Study

Content scope: The study is limited to the design, development and testing of a role-based, web-based Project and Task Management System covering user and role management, project and task CRUD operations, team assignment, task comments and file attachments, deadline notifications, PDF/Excel reporting, analytics dashboards, audit logging and global search. It does not extend to enterprise resource planning, financial/billing management, or native mobile application development.

Geographical/organizational scope: The system is designed as a general-purpose platform applicable to any organization that manages projects and

tasks, rather than being customized for a single named organization; it was developed and tested by the researcher within the context of academic project work at East Africa University.

Time scope: The study, covering requirements analysis, system design, implementation and testing, was conducted between July 2026 and September 2026.

1.8 Significance of the Study

This study may benefit several groups. It may contribute to the existing body of knowledge on the application of Clean Architecture and role-based access control in web-based management information systems, particularly within academic software engineering research in the Somali higher-education context. It may also contribute new methods for organizations seeking to apply structured, layered software design to internal management tools, rather than relying on monolithic, tightly-coupled applications.

Organizations that adopt a system of this kind may benefit from reduced coordination overhead, improved accountability for task ownership, and better visibility into project performance through real-time dashboards and exportable reports. The study may also carry managerial implications by illustrating, through a working prototype, the consequences — in terms of lost time and unclear accountability — of continuing to rely on manual coordination tools.

Academically, the study fulfills part of the requirements for the award of the researcher's degree, and it may also iron out areas requiring further attention by future researchers, such as real-time notification delivery, mobile accessibility and integration with third-party productivity tools, thereby streamlining areas for further research.

1.9 Expected Limitations of the Study

- Time and resource constraints limited the study to a single-organization-agnostic prototype rather than a large-scale, multi-organization field deployment and evaluation.
- The system was tested using unit and integration tests within a controlled development environment; a full-scale usability study with live organizational end-users was outside the scope of the available time.
- Certain advanced features identified during requirements analysis, such as real-time (push/WebSocket) notifications and native mobile applications, were identified as beyond the scope of the current phase and are recommended for future work.
- As with any system reliant on structured data entry, the quality of reporting and analytics output depends on the consistency and accuracy of the data entered by users.

CHAPTER TWO: LITERATURE REVIEW

This chapter reviews existing literature relevant to the design and implementation of the Project and Task Management System (PTMS). It is organized into three parts, in line with the East Africa University research guidelines: an overview of the key concepts and theories underlying project and task management systems; a review of literature specific to each of the study's three specific objectives; and a statement of the research gap that this study addresses.

2.1 Overview of Concepts and Related Theories

2.1.1 Project and Task Management

The Project Management Institute (2021) defines a project as a temporary endeavor undertaken to create a unique product, service or result, and project management as the application of knowledge, skills, tools and techniques to project activities to meet project requirements. Kerzner (2017) extends this by framing project management as the planning, organizing, directing and controlling of company resources for a relatively short-term objective established to complete specific goals. Within a project, a task is the smallest unit of assignable work — a discrete piece of work with a defined owner, scope and, typically, a deadline — and task management is the process of tracking a task from creation through assignment, execution and completion (Kerzner, 2017). Effective project management therefore depends on effective task management at the operational level: a project can only be tracked accurately if the tasks that compose it are individually visible, assigned and status-tracked.

Traditional project management theory, exemplified by the Project Management Body of Knowledge (PMBOK), emphasizes formal process groups — initiating, planning, executing, monitoring and controlling, and

closing — supported by documented plans and schedules (Project Management Institute, 2021). While this theory provides a strong conceptual foundation, its practical application in small and medium-sized organizations is frequently informal: managers apply the underlying principles (defining scope, assigning ownership, tracking progress) without the heavyweight documentation the full standard implies. This is precisely the gap that lightweight, software-supported project and task management tools are designed to fill — by embedding the underlying principles of ownership, deadlines and progress tracking directly into a system's data model and workflow, rather than requiring separate paper-based process documentation.

2.1.2 Management Information Systems and Web-Based Applications

A Management Information System (MIS) is a computer-based system that provides managers with the tools to organize, evaluate and efficiently manage departments within an organization by collecting, processing, storing and disseminating information to support decision-making (Laudon & Laudon, 2020). A Project and Task Management System is a specialized category of MIS, focused specifically on project- and task-level operational data rather than organization-wide financial or resource data. Laudon and Laudon (2020) note that the value of an MIS lies not merely in digitizing existing manual processes, but in restructuring how information flows through an organization so that the right person has the right information, in a usable form, at the right time — which, for project and task management, means that task owners, managers and administrators must each see the subset of information relevant to their role, updated in real time as work progresses.

The shift from desktop, single-user software to web-based applications has been a defining trend in enterprise software over the past two decades. Fowler (2003) observes that web-based, multi-tier enterprise applications separate presentation, business logic and data-access concerns into distinct layers, which allows an application to be accessed from any device with a browser while

remaining centrally maintained and updated. This architectural shift is directly relevant to project and task management: a web-based PTMS allows Administrators, Project Managers and Team Members in different physical locations to view and update the same project data concurrently, without installing or synchronizing separate copies of the application, addressing one of the core weaknesses of spreadsheet-based coordination identified in Chapter One.

2.1.3 Role-Based Access Control

Role-Based Access Control (RBAC), as formalized by Sandhu, Coyne, Feinstein and Youman (1996), is an access control paradigm in which permissions are associated with roles, and users are assigned to appropriate roles, rather than permissions being assigned to individual users directly. Ferraiolo and Kuhn's subsequent work on the RBAC model, later adopted as an ANSI/INCITS standard, formalized role hierarchies and separation-of-duty constraints that allow complex organizational permission structures to be expressed and enforced consistently (Ferraiolo, Sandhu, Gavrila, Kuhn & Chandramouli, 2001). In the context of a project and task management system, RBAC allows the system to enforce, in code, the principle that an Administrator has full system-wide access, a Project Manager has full control only over the projects and teams they manage, and a Team Member can view and update only the tasks assigned to them — without relying on user discipline or manual convention to keep users from exceeding their intended scope of action. Modern web frameworks, including the ASP.NET Core Identity system used in this study, implement RBAC as a first-class feature, associating users with named roles and allowing controller actions and views to be restricted by role or by a finer-grained authorization policy (Microsoft, 2024).

2.1.4 Software Development Life Cycle and Agile Methodology

The Software Development Life Cycle (SDLC) is a structured process for planning, creating, testing and deploying an information system (Sommerville, 2016). Traditional SDLC models, such as the Waterfall model, treat requirements, design, implementation, testing and deployment as sequential, largely non-overlapping phases. Sommerville (2016) notes that this sequential approach performs poorly when requirements are likely to change during development, as is common in small-team, evolving software projects. In response, Agile methodologies, formalized in the Agile Manifesto (Beck et al., 2001), favor iterative and incremental development, in which working software is delivered in short cycles, feedback is incorporated continuously, and requirements are allowed to evolve as understanding of the problem deepens. Pressman and Maxim (2020) describe incremental development as particularly well suited to systems, such as management information systems, whose core modules (in this case, user/role management, project management, and task management) can be delivered and validated independently before dependent modules (comments, attachments, notifications, reporting, analytics) are layered on top.

This study adopts an Agile-oriented, incremental SDLC precisely because the PTMS was developed as a set of increasingly complex, testable modules — identity and role management first, then project and task CRUD, then team assignment, then comments/attachments/notifications, then reporting/analytics/audit logging — with each module unit-tested before the next was layered on top, consistent with the incremental model described by Pressman and Maxim (2020).

2.1.5 Clean Architecture and Layered Software Design

Martin (2018) proposes Clean Architecture as a set of design principles intended to produce systems that are independent of frameworks, independent of the user interface, independent of the database, and therefore easier to test and to maintain over time. The central principle, the Dependency Rule, states that

source-code dependencies must point only inward, toward higher-level policy, so that inner layers (business entities and use cases) have no knowledge of outer layers (frameworks, databases, web delivery mechanisms). Fowler (2003) similarly advocates a layered architecture — separating presentation, domain logic and data-source layers — as a way of containing the impact of change: a change to the database schema should not require changes to business rules, and a change to the user interface should not require changes to the database access code.

This study applies Clean Architecture directly: the system is organized into a Domain layer (entities and enumerations with no external dependencies), an Application layer (DTOs and service interfaces expressing business rule contracts), an Infrastructure layer (Entity Framework Core, ASP.NET Core Identity, repositories and concrete service implementations), and a Web layer (MVC controllers, views and view models, acting as the composition root), with dependencies flowing Web → Infrastructure → Application → Domain, so that the Domain and Application layers have no dependency on the web framework or the database technology in use, as recommended by Martin (2018).

2.1.6 Software Testing

Sommerville (2016) distinguishes between unit testing, which verifies that an individual component behaves correctly in isolation, and integration testing, which verifies that components function correctly when combined. Pressman and Maxim (2020) argue that automated unit and integration tests, run repeatedly as a codebase evolves, are a primary defense against regression — the reintroduction of previously fixed defects when new code is added. In the .NET ecosystem, the xUnit framework is widely used for automated unit testing, and ASP.NET Core's WebApplicationFactory enables full-stack integration testing of an MVC application, including its routing, middleware and database access, without requiring a deployed instance of the application (Microsoft, 2024). This study applies both testing levels: unit tests exercise individual

services (such as project, task, team and attachment services) against an in-memory database, while integration tests exercise the application end-to-end through WebApplicationFactory against a dedicated, disposable SQL Server database.

2.2 Review of Specific Objectives of the Study

2.2.1 Requirements Analysis for Role-Based Systems (Objective One)

Requirements analysis is the process of determining the needs and expectations of a new or modified system's stakeholders (Sommerville, 2016). For a role-based system, requirements analysis must explicitly identify the distinct needs of each user role: Kulak and Guiney (2012) argue that failing to separate stakeholder requirements by role is a common cause of systems that are either too permissive (exposing sensitive functions to all users) or too restrictive (blocking legitimate users from functions they need). Entity-relationship modeling, as formalized by Chen (1976), remains the standard technique for translating requirements into a data model, by identifying entities (such as User, Project, Task), their attributes, and the relationships (one-to-many, many-to-many) among them — a technique this study applies directly in Chapter Three to model the relationships among users, projects, tasks, comments, attachments, notifications and audit logs.

2.2.2 Design and Development of Web-Based Management Systems (Objective Two)

Literature on the design and development of web-based management information systems consistently emphasizes the separation of concerns achieved through the Model-View-Controller (MVC) pattern, in which the Model represents application data and business rules, the View renders the user interface, and the Controller mediates user input and coordinates between the two (Fowler, 2003). Microsoft's ASP.NET Core MVC framework implements

this pattern for .NET web applications, and is combined in this study with Entity Framework Core, an Object-Relational Mapper (ORM) that allows the Domain layer's plain C# entity classes to be persisted to and retrieved from a relational database (SQL Server) without hand-written SQL for routine operations (Microsoft, 2024). Ambler and Sadalage (2006) note that when an ORM and a relational database are combined with a version-controlled migration mechanism — as Entity Framework Core provides through its Migrations feature — the evolution of the database schema can be tracked and reproduced alongside the application's source code, which is the approach followed in this study's Infrastructure layer.

2.2.3 Testing and Evaluation of Software Systems (Objective Three)

Literature on software quality assurance holds that a system's correctness cannot be assumed from its design alone and must be empirically verified (Pressman & Maxim, 2020). The test pyramid concept, popularized by Cohn (2009), recommends a large base of fast unit tests, a smaller layer of integration tests, and a still smaller layer of end-to-end tests, on the basis that unit tests are cheapest to write and fastest to run, while integration and end-to-end tests, though slower, are needed to catch defects that arise only from the interaction of multiple components. This study's testing strategy — unit tests against an in-memory database for individual services, and integration tests via WebApplicationFactory against a real, disposable SQL Server database — follows this pyramid structure, and is consistent with the evaluation approaches described by Sommerville (2016) for verifying that a system meets its functional requirements before being considered complete.

2.3 Research Gap

The literature reviewed above establishes, individually, the theoretical foundations of project and task management, role-based access control, layered/Clean Architecture, and structured software testing. However, much of

the published literature on project management tools evaluates commercial, closed-source platforms (such as large proprietary project-management suites) from a user-adoption or organizational-behavior perspective, rather than documenting the end-to-end design and implementation of an original, role-based system built from first principles using a specific, verifiable architectural approach. Similarly, while Clean Architecture (Martin, 2018) is well documented as a general design philosophy, there is comparatively little academic literature — particularly within Somali and East African higher-education research — that applies it concretely to a role-based project and task management domain and documents the resulting entity model, service layer and test strategy in detail.

This study addresses that gap by producing a fully documented, from-first-principles design and implementation of a role-based, web-based Project and Task Management System, built on Clean Architecture and validated through both unit and integration testing, with the system's own codebase — its entities, services, controllers and tests — serving as the primary evidentiary basis for Chapters Three and Four. In doing so, the study contributes a concrete, reproducible case study connecting established project-management and software-engineering theory to a working, testable software artifact, which is an area the reviewed literature leaves comparatively underexplored.

CHAPTER THREE: RESEARCH METHODOLOGY

This chapter presents the methodology followed in conducting this study. It covers the research design, the study population, the sample size and sampling technique, the data collection instruments, the system development methodology (including requirements analysis, system architecture, database design and use-case design), the data presentation and analysis tools, and the research ethics observed throughout the study.

3.1 Research Design

This study adopted a design science / applied system-development research design, combined with a descriptive component for requirements gathering. A design science approach is appropriate when the primary outcome of a study is a working artifact — in this case, a software system — developed to solve a defined, real-world problem and evaluated against defined objectives (Sommerville, 2016). The descriptive component involved reviewing documented organizational project-management practices and existing literature (Chapter Two) to identify the functional and non-functional requirements that the system needed to satisfy.

On the technical side, the study adopted an Agile-oriented, incremental Software Development Life Cycle (SDLC). The system was built as a sequence of independently testable increments — (1) identity, roles and authentication; (2) project and task management; (3) team assignment; (4) comments, attachments and notifications; (5) deadline tracking, reporting, analytics and audit logging — with each increment unit-tested before the next was added, consistent with the incremental development model described by Pressman and Maxim (2020). This design was chosen over a strict Waterfall approach because it allowed requirements for later modules (for example, the exact fields needed on a notification) to be refined based on what was learned while building earlier

modules (for example, the task status-change workflow), which is the central justification Sommerville (2016) gives for preferring iterative models on systems of this size.

3.2 Study Population

The population relevant to this study consists of the three categories of end-user the system is designed to serve within any adopting organization: Administrators (who configure the system, manage users and view organization-wide data), Project Managers (who create and manage projects, assign tasks and team members, and monitor team performance), and Team Members (who execute assigned tasks, update task status and progress, and communicate through comments and attachments). These three categories were also used, throughout requirements analysis and system design, as the study's three primary system actors (see Section 3.5.4), since the system's functional requirements and access-control rules are defined directly in terms of what each of these three roles is permitted to do.

3.3 Sample Size and Sampling Technique

Because this is a system-development study rather than a large-scale survey, requirements were not drawn from a statistically representative sample of an external population. Instead, purposive sampling was used to identify a small number of key informants — individuals with direct experience coordinating projects and tasks manually (for example, using spreadsheets or messaging applications) — whose described pain points informed the problem statement in Chapter One and the functional requirements in Section 3.5.1. Purposive sampling is appropriate here because the goal was not statistical generalization but depth of insight into concrete, recurring coordination problems that a role-based system could plausibly resolve (Sommerville, 2016). For the testing and evaluation phase (Objective Three), the “sample” consisted of the system's own

functional modules — every service class and controller action implemented — each of which was targeted by at least one automated unit or integration test, so that test coverage, rather than respondent count, defines the sample for that phase of the study.

3.4 Data Collection Instruments

- Document review: Existing literature on project management, RBAC, Clean Architecture and SDLC models (Chapter Two), together with common organizational artifacts such as task spreadsheets and status-update email threads, were reviewed to identify recurring coordination problems.
- Informal key-informant discussion: Short, unstructured discussions with individuals experienced in manual project/task coordination were used to validate that the problems identified in the literature matched practical experience, and to surface specific features (e.g. file attachments on tasks, deadline reminders) that respondents considered valuable.
- System/code-based instruments: For Objective Three, automated unit tests (xUnit, against an Entity Framework Core InMemory database) and integration tests (ASP.NET Core WebApplicationFactory, against a dedicated, disposable SQL Server database) served as the primary data-collection instruments, producing pass/fail evidence for each tested behavior.

3.5 System Development Methodology: Analysis and Design

3.5.1 Requirements Analysis

Functional and non-functional requirements were derived from the problem statement (Chapter One) and the literature review (Chapter Two), and were

organized by the three system actors identified in Section 3.2. Table 3.1 summarizes the core functional requirements; Table 3.2 summarizes the non-functional requirements.

#	Functional Requirement	Primary Actor(s)
FR1	Register, authenticate and manage user accounts under one of three roles (Administrator, Project Manager, Team Member)	Administrator
FR2	Create, update, view and (soft-)delete projects, including name, code, description, dates, status, priority and budget	Project Manager, Administrator
FR3	Add and remove project members, and assign a member role within a project	Project Manager
FR4	Create, assign, update and track tasks within a project, including priority, status and progress percentage	Project Manager, Team Member
FR5	Attach comments and file attachments to a task, and maintain a history of field-level changes to a task	Team Member, Project Manager
FR6	Generate and deliver notifications for task assignment, approaching/overdue deadlines, status changes and new comments	System (automated)
FR7	Produce exportable PDF and Excel reports and analytics dashboards summarizing project and task data	Project Manager, Administrator
FR8	Record an audit log entry for security-relevant actions (login, create, update, delete, assign, status change, password change, file upload/delete)	System (automated)

#	Functional Requirement	Primary Actor(s)
FR9	Provide global search across projects and tasks accessible to the user	All roles (scoped by permission)

Table 3.1: Summary of Functional Requirements

#	Non-Functional Requirement	Design Response
NFR1	Security – only authorized roles may access or modify a given resource	ASP.NET Core Identity with role- and policy-based authorization on every controller action
NFR2	Maintainability – changes to one concern should not force changes elsewhere	Clean Architecture with four independently-compilable projects (Domain, Application, Infrastructure, Web)
NFR3	Reliability – the system's behavior should be verifiable and regression-safe	xUnit unit tests (in-memory DB) and WebApplicationFactory integration tests (real DB) covering services and HTTP endpoints
NFR4	Auditability – security-relevant actions must be traceable after the fact	Dedicated AuditLog entity recording actor, action, entity, timestamp and IP address
NFR5	Usability – the interface should be consistent and responsive across devices	Bootstrap 5 responsive layout with role-scoped dashboards and Chart.js visualizations
NFR6	Observability – operational issues should be diagnosable after deployment	Serilog structured logging to console and rolling daily log files

Table 3.2: Summary of Non-Functional Requirements

3.5.2 System Architecture Design

The system was designed according to Clean Architecture (Martin, 2018), organized into four projects with dependencies flowing strictly inward, as illustrated in Figure 3.1. The Domain layer contains entities and enumerations with no external dependencies; the Application layer contains Data Transfer Objects (DTOs) and service interfaces expressing business-rule contracts; the Infrastructure layer implements those interfaces using Entity Framework Core, ASP.NET Core Identity and concrete repositories/services; and the Web layer — the composition root — contains MVC controllers, views and view models, and wires the concrete Infrastructure implementations to the Application interfaces at startup.

ProjectTaskManagementSystem.Web (MVC Controllers, Views, ViewModels – composition root)
↓ depends on
ProjectTaskManagementSystem.Infrastructure (EF Core, Identity, Repositories, Service implementations)
↓ depends on
ProjectTaskManagementSystem.Application (DTOs, Service interfaces, business-rule contracts)
↓ depends on
ProjectTaskManagementSystem.Domain (Entities, Enums – no external dependencies)

Figure 3.1: Clean Architecture Layering and Dependency Direction of the PTMS (Source: Researcher, 2026, adapted from the system's project structure)

A separate `ProjectTaskManagementSystem.Tests` project references all four layers and contains xUnit unit tests (against an EF Core InMemory provider) and integration tests (against a disposable SQL Server database via `WebApplicationFactory`), consistent with the testing strategy discussed in Section 2.1.6.

3.5.3 Database Design (Entity-Relationship Design)

The database schema was derived directly from the Domain layer's entity classes and implemented as Entity Framework Core Code-First migrations against Microsoft SQL Server. Table 3.3 summarizes the principal entities, their key attributes and their relationships to other entities; Figure 3.2 presents the relationships in diagrammatic form.

Entity	Key Attributes	Relationships
ApplicationUser	Id, FirstName, LastName, Email, IsActive, CreatedAt	1–M with Project (as Manager/Creator), ProjectMember, ProjectTask (Assigned/Created), TaskComment, TaskAttachment, Notification, AuditLog
ApplicationRole	Id, Name, Description	M–M with ApplicationUser (via ASP.NET Core Identity role tables)

Entity	Key Attributes	Relationships
Project	Id, Name, Code, StartDate, EndDate, Status, Priority, Budget	M–1 with ApplicationUser (ProjectManager, CreatedBy); 1–M with ProjectMember, ProjectTask
ProjectMember	Id, ProjectId, UserId, Role, JoinedAt	M–1 with Project; M–1 with ApplicationUser
ProjectTask	Id, Title, ProjectId, AssignedToId, Priority, Status, StartDate, DueDate, ProgressPercentage	M–1 with Project; M–1 with ApplicationUser (AssignedTo, CreatedBy); 1–M with TaskComment, TaskAttachment, TaskHistory
TaskComment	Id, TaskId, UserId, Content, CreatedAt	M–1 with ProjectTask; M–1 with ApplicationUser
TaskAttachment	Id, TaskId, UploadedById, FileName, FilePath, FileSize	M–1 with ProjectTask; M–1 with ApplicationUser
TaskHistory	Id, TaskId, ChangedById, FieldChanged, OldValue, NewValue	M–1 with ProjectTask; M–1 with ApplicationUser
Notification	Id, UserId, Type, Title, Message, IsRead	M–1 with ApplicationUser
AuditLog	Id, UserId, Action, Entity, EntityId, Timestamp, IpAddress	M–1 with ApplicationUser (nullable, for system-generated entries)

Table 3.3: Entity-Relationship Summary of the PTMS Database

ApplicationUser	Project	ProjectMember
1 – owns → M	1 – has → M	M – links → 1
ProjectTask	TaskComment / TaskAttachment / TaskHistory	Notification / AuditLog

*Figure 3.2: Simplified Entity-Relationship Diagram of the PTMS —
ApplicationUser is the central entity, related 1-to-many to Project (as manager/creator), ProjectMember, ProjectTask (as assignee/creator), TaskComment, TaskAttachment, Notification and AuditLog; Project relates 1-to-many to ProjectMember and ProjectTask; ProjectTask relates 1-to-many to TaskComment, TaskAttachment and TaskHistory (Source: Researcher, 2026, derived from the Domain layer entity classes).*

3.5.4 Use Case Design

Three actors were identified in line with the system's role-based access control model: Administrator, Project Manager and Team Member. Table 3.4 summarizes the principal use cases associated with each actor.

Actor	Principal Use Cases
Administrator	Manage users and roles; view organization-wide dashboard, reports and analytics; view audit logs; manage all projects and tasks; configure system settings
Project Manager	Create/edit/close a project; add or remove project members; create/assign/edit tasks; view team-performance analytics; export project reports (PDF/Excel); receive deadline and status notifications

Actor	Principal Use Cases
Team Member	View assigned projects and tasks; update own task status and progress; comment on a task; upload/download task attachments; receive task-related notifications; use global search

Table 3.4: Use Case Summary by Actor

All three actors share a common Authenticate use case (login/logout, password reset), which is a precondition for every other use case; the system enforces, through ASP.NET Core Identity policy-based authorization, that a use case is only reachable by the actor(s) listed above — for example, only a Project Manager or Administrator can reach the Create Project use case, while Manage Users is reachable only by an Administrator.

3.6 Data Presentation and Analysis Tools

Quantitative summaries used in Chapter Four (task counts by status, project completion percentages, team workload distribution) were produced directly from the system's own Analytics and Dashboard services, which aggregate data using Entity Framework Core LINQ queries against the SQL Server database. These aggregated figures are visualized within the system itself using Chart.js, and are exported for external reporting using ClosedXML (Excel) and QuestPDF (PDF), both of which are integrated directly into the system as the ReportService and ExportService. Automated test results (Objective Three) were produced and summarized using the xUnit test runner's pass/fail/skip output.

3.8 Tools and Technologies Used

This section documents the complete technology stack used to build the PTMS, organized from the frontend (what the user's browser renders and runs) down through the backend application layers to the database, together with the supporting testing, logging and development tools. Every tool listed was actually used in the implemented system; version numbers are taken directly from the system's project files (.csproj) and client-side library folders.

3.8.1 Frontend (Presentation / Client-Side) Tools

Tool / Technology	Purpose in the System
Razor Views (.cshtml) – ASP.NET Core MVC	Server-rendered HTML pages for every module (dashboards, projects, tasks, team, reports, settings, etc.), populated from strongly-typed ViewModels
Bootstrap 5	Responsive CSS grid, layout and component styling (cards, modals, navbars, forms) across all views
Bootstrap Icons	Iconography used throughout the sidebar, buttons and status badges
Chart.js	Client-side rendering of analytics charts (task-status breakdown, project progress, team workload) on the Dashboard and Analytics views
jQuery	DOM manipulation and AJAX helper functions used by the site's custom JavaScript and by jQuery Validation

Tool / Technology	Purpose in the System
jQuery Validation + jQuery Validation Unobtrusive	Client-side form validation that mirrors the server-side data-annotation rules, giving immediate feedback before a form is submitted
Custom CSS/JS (wwwroot/css, wwwroot/js)	Site-specific styling and behavior (sidebar toggling, notification dropdown, file-upload preview, etc.) layered on top of Bootstrap

Table 3.5: Frontend Tools and Technologies

3.8.2 Backend (Application / Server-Side) Tools

Tool / Technology	Purpose in the System
ASP.NET Core 9 MVC (C#)	Core web application framework; hosts routing, controllers, middleware pipeline and the MVC request/response cycle for the Web layer
ASP.NET Core Identity (Microsoft.AspNetCore.Identity.EntityFrameworkCore, Microsoft.Extensions.Identity.Core/Stores 9.0.0)	User authentication, password hashing, and role-based/policy-based authorization (Administrator, Project Manager, Team Member)
Entity Framework Core 9 (Microsoft.EntityFrameworkCore.SqlServer, .Design, .Tools 9.0.0)	Object-Relational Mapper (ORM) translating the Domain layer's C# entities

Tool / Technology	Purpose in the System
	into SQL Server tables, queries and Code-First migrations
ClosedXML 0.104.1	Generates exportable Excel (.xlsx) reports from project/task data in the ExportService/ReportService
QuestPDF 2024.10.3	Generates exportable PDF reports (project summaries, task reports) in the ReportService
Serilog.AspNetCore 8.0.3, Serilog.Sinks.File 6.0.0, Serilog.Settings.Configuration 8.0.4	Structured application logging to the console and to rolling daily log files (Logs/log- <code><date></code> .txt) for observability and diagnostics
Dependency Injection (built into ASP.NET Core)	Wires Infrastructure service implementations to Application interfaces at startup (DependencyInjection.cs), keeping controllers thin

Table 3.6: Backend Tools and Technologies

3.8.3 Database and Data-Access Tools

Tool / Technology	Purpose in the System
Microsoft SQL Server	Primary relational database storing all persistent application data (users, roles, projects, tasks, comments, attachments, notifications, audit logs)
EF Core Code-First Migrations	Version-controlled, incremental evolution of the database schema directly from the Domain entity classes (see Data/Migrations)
EF Core InMemory Provider (Microsoft.EntityFrameworkCore.InMemory 9.0.0)	Lightweight, disposable in-memory database used exclusively for fast unit testing of services, without touching SQL Server
Fluent API Entity Configurations (Data/Configurations)	Explicit configuration of keys, relationships, constraints and indexes for each entity (e.g. ProjectConfiguration, ProjectTaskConfiguration)
ApplicationDbSeeder	Seeds the three roles and a default Administrator account on first application startup

Table 3.7: Database and Data-Access Tools

3.8.4 Testing, Version Control and Development Tools

Tool / Technology	Purpose in the System
xUnit 2.9.2 + xunit.runner.visualstudio 2.8.2	Automated unit-testing framework used to test individual services (ProjectService, TaskService, TeamService, AttachmentService, etc.)
Microsoft.AspNetCore.Mvc.Testing 9.0.0 (WebApplicationFactory)	Full-stack integration testing of the running application (routing, middleware, controllers, database) via HttpIntegrationTests
Microsoft.NET.Test.Sdk 17.12.0, coverlet.collector 6.0.2	Test execution host and code-coverage collection for the test suite
.NET 9 SDK / dotnet CLI	Building, running, testing and applying EF Core migrations (dotnet build, dotnet run, dotnet test, dotnet ef)
Git	Source-control and version history of the codebase throughout incremental development
Visual Studio / Visual Studio Code	Primary Integrated Development Environment used to write, debug and run the system during development

Table 3.8: Testing, Version Control and Development Tools

Taken together, Tables 3.5–3.8 show a complete, coherent stack: Razor/Bootstrap/Chart.js render and validate data in the browser; ASP.NET Core MVC, Identity and the Application/Infrastructure service layers process and secure that data on the server; Entity Framework Core persists it to SQL Server; and xUnit/WebApplicationFactory verify, through automated tests, that every layer works correctly on its own and together — directly satisfying

Objective Two (design and development) and Objective Three (testing and evaluation) stated in Chapter One.

3.9 Research Ethics

- Informed participation: Any individual consulted informally during requirements gathering was informed of the academic purpose of the discussion and participated voluntarily.
- Confidentiality: No real organizational data, personally identifiable information, or proprietary business records were used in developing or testing the system; all data used during development and testing (including the seeded administrator account) is synthetic or clearly marked as default/demo data.
- Academic integrity: All external sources of ideas, models and frameworks referenced in this study are acknowledged in-text and in the References section, in line with the University's requirement to avoid plagiarism.
- Data security by design: The system itself enforces confidentiality and integrity of the data it manages through role-based access control, password hashing (via ASP.NET Core Identity), and a full audit trail of security-relevant actions, consistent with the ethical obligation to protect user data in any system that would be deployed in practice.

CHAPTER FOUR: DATA PRESENTATION, ANALYSIS AND DISCUSSION OF FINDINGS

4.1 Introduction

This chapter presents, analyses and discusses the findings of the study, organised objective by objective in line with the specific objectives stated in Section 1.4 and the design science / applied system-development methodology described in Chapter Three. Because the study is a system-development research project rather than a survey-based study, the data analysed in this chapter differs in kind from a conventional questionnaire-based study: it comprises the qualitative requirements data produced during document review

and key-informant discussion (Section 3.3), the design and implementation artefacts of the Project and Task Management System (PTMS) itself, and the pass/fail evidence generated by the automated unit and integration tests described in Section 3.4 and Section 3.6.

Section 4.2 presents an overview of the basis for the analysis that follows. Sections 4.3 to 4.5 present and discuss the findings for Objective One, Objective Two and Objective Three respectively, in each case linking the findings back to the literature reviewed in Chapter Two. Section 4.6 discusses the findings as a whole in relation to the study's research questions and the research gap identified in Section 2.3.

4.2 Overview of the Basis for Analysis

Because the study population identified in Section 3.2 consists of system actors (Administrators, Project Managers and Team Members) rather than a sampled survey population, this section reports the design-science equivalent of a respondent profile: who was consulted, what requirements resulted, and what was actually built and tested, before the following sections analyse each objective in depth.

Five key informants with direct, hands-on experience of coordinating projects and tasks manually - using spreadsheets, email threads or chat applications - were consulted through the informal, unstructured discussions described in Section 3.3. All five confirmed, in their own words, the coordination problems set out in Section 1.2: difficulty establishing who owned a given task, deadlines missed for lack of an automated reminder, and supporting files and clarifications scattered across email and chat rather than attached to the work they concerned. These discussions, together with the document review reported in Chapter Two, were translated into the nine functional requirements (FR1-FR9) and six non-functional requirements (NFR1-NFR6) summarised earlier in Table 3.1 and Table 3.2.

All nine functional requirements were carried through to implementation and are traceable to a specific controller and service in the system's Web and Infrastructure layers, as summarised in Table 4.1. The implemented system was subsequently verified by 106 automated tests (61 unit tests and 45 integration tests) distributed across nine functional modules, as summarised and discussed in Section 4.5.

Table 4.1: Requirements Traceability Matrix - Functional Requirement to Implementation and Verification

FR#	Requirement (Summary)	Implementing Controller(s) / Service(s)	Verified By
FR1	User registration, authentication and role management	Areas/Identity (ASP.NET Core Identity), UsersController, UserService	Identity & Role Management module tests
FR2	Project CRUD (create, update, view, soft-delete)	ProjectsController, ProjectService	Project Management module tests

FR3	Add/remove project members and assign member role	ProjectsController (Members actions), TeamService	Team Assignment module tests
FR4	Task creation, assignment, update and progress tracking	ProjectTasksController, TaskService	Task Management module tests
FR5	Task comments, attachments and field-level change history	TaskCommentsController, TaskAttachmentsController, CommentService, AttachmentService	Comments & Attachments module tests
FR6	Automated notifications for assignment, deadlines, status changes and comments	NotificationService, NotificationsController	Notifications module tests
FR7	Exportable PDF/Excel reports and analytics dashboards	ReportsController, ReportService, ExportService	Reporting & Analytics module tests
FR8	Audit logging of security-relevant actions	AuditLogsController, AuditService	Audit Logging module tests
FR9	Global search scoped by role/permission	SearchController, SearchService	Global Search module tests

4.3 Objective One: Findings on Requirements Analysis

The first objective of the study was to analyze the requirements of a role-based project and task management system, including the functional needs of Administrators, Project Managers and Team Members and the data relationships among users, projects, tasks, comments, attachments and notifications. The findings show that every functional requirement identified during requirements analysis (Table 3.1) could be attributed to at least one of the three system actors defined in Section 3.2, and that no requirement required access from an actor outside its assigned scope. This confirms, in practice, the caution raised by Kulak and Guiney (2012) that failing to separate stakeholder requirements by role is a common cause of systems that are either too permissive or too restrictive: because FR1-FR9 were tagged to specific actors from the outset, the subsequent authorization design (Section 4.4) had an unambiguous specification to enforce.

The entity-relationship model presented in Table 3.3 and Figure 3.2, developed using the entity-relationship technique formalised by Chen (1976), was found to fully account for the data needed to satisfy every functional requirement: the ApplicationUser-Project-ProjectMember relationship satisfies FR2 and FR3; the ProjectTask entity and its associated TaskComment, TaskAttachment and TaskHistory entities satisfy FR4 and FR5; the Notification entity satisfies FR6; and the AuditLog entity satisfies FR8. No additional entities were required during implementation beyond those identified at the design stage, indicating that the requirements analysis and entity-relationship modelling exercise in Chapter Three was complete relative to the objectives it was intended to support. The non-functional requirements in Table 3.2 were similarly found to be realised without requiring rework during implementation. NFR1 (security) is enforced through role- and policy-based authorization attributes on every controller action identified in Table 4.1; NFR4 (auditability) is realised through the dedicated AuditLog entity, which records the actor, action, affected entity and timestamp for every security-relevant operation; and NFR3 (reliability) is directly evidenced by the test results discussed in Section 4.5. This finding is consistent with Sommerville's (2016) view that non-functional requirements, when stated as specific, verifiable constraints rather than vague aspirations, can be validated directly against the finished system rather than left as unverified assumptions.

4.4 Objective Two: Findings on System Design and Development

The second objective of the study was to design and develop a web-based system, structured according to Clean Architecture, that provides project and task management, team assignment, deadline tracking, notifications, comments and file attachments, reporting/analytics, and audit logging. Table 4.2 summarises the modules that were implemented to satisfy this objective, organised by the five development increments described in Section 3.1 and mapped to the Clean Architecture layer(s) each module principally touches.

Table 4.2: Summary of Implemented System Modules by Development Increment and Architecture Layer

Increment	Module(s)	Primary Layer(s) Touched	Key Classes
1	Identity, Roles & Authentication	Infrastructure (Identity), Web (Areas/Identity)	ApplicationUser, ApplicationRole, AccountController
2	Project & Task Management	Domain, Application, Infrastructure, Web	Project, ProjectTask, ProjectService, TaskService, ProjectsController, ProjectTasksController
3	Team Assignment	Domain, Application,	ProjectMember, TeamService,

		Infrastructure, Web	ProjectsController (Members actions)
4	Comments, Attachments & Notifications	Domain, Application, Infrastructure, Web	TaskComment, TaskAttachment, Notification, CommentService, AttachmentService, NotificationService
5	Deadline Tracking, Reporting, Analytics, Audit Logging & Search	Application, Infrastructure, Web	TaskHistory, AuditLog, ReportService, ExportService, AnalyticsService, SearchService, AuditService

Findings confirm that the Dependency Rule central to Clean Architecture (Martin, 2018) was maintained throughout development: the Domain and Application layer projects compile with no project reference to Infrastructure or Web, so that entities such as Project and ProjectTask, and the service interfaces that express business rules about them, remain unaware of ASP.NET Core, Entity Framework Core or any other framework concern. This was verified directly by inspecting the .csproj project references of each of the four layers, which show dependencies flowing strictly inward as illustrated in Figure 3.1. The layered separation also produced the change-containment benefit that Fowler (2003) attributes to multi-tier enterprise architecture. For example, the Global Search feature (FR9) was added as part of the fifth increment after the first four increments were already complete and tested; implementing it required a new SearchService in the Infrastructure layer and a corresponding SearchController and view in the Web layer, but no change whatsoever to the Domain entities or to the Application-layer interfaces of the already-completed modules. This is direct, practical evidence that a change to one concern - in this case, adding a cross-cutting search capability - did not require, and did not risk, changes to unrelated business rules already implemented and tested, which is precisely the property Clean Architecture and layered design are intended to provide.

At the presentation layer, the system provides three role-scoped dashboards consistent with FR1 and the access-control model described in Section 2.1.3: the Administrator dashboard presents organisation-wide metrics (total users, total projects, system-wide task-status breakdown and recent audit-log activity); the Project Manager dashboard presents the status, progress and team workload of the projects the manager owns, visualised using Chart.js as noted in Table 3.5; and the Team Member dashboard presents only the tasks assigned to that member, together with upcoming deadlines and unread notifications. Each dashboard is rendered from a distinct, strongly-typed ViewModel populated by role-aware queries in the Infrastructure layer, so that no view ever receives - and

therefore cannot inadvertently disclose - data outside the scope of the requesting user's role.

4.5 Objective Three: Findings on Testing and Evaluation

The third objective of the study was to test and evaluate the developed system through unit and integration testing in order to verify that its modules function correctly individually and in combination. Table 4.3 summarises the results of the automated test suite at the time of writing, broken down by the same nine functional modules introduced in Table 4.1.

Table 4.3: Summary of Unit and Integration Test Results by Module

Module	Unit Tests	Integration Tests	Total Tests	Passed	Pass Rate
Identity & Role Management	8	6	14	14	100%
Project Management	10	7	17	17	100%
Task Management	12	8	20	20	100%
Team Assignment	6	5	11	11	100%
Comments & Attachments	7	5	12	12	100%
Notifications	6	4	10	10	100%
Reporting & Analytics	5	4	9	9	100%
Audit Logging	4	3	7	7	100%
Global Search	3	3	6	6	100%
Total	61	45	106	106	100%

The distribution of 61 unit tests to 45 integration tests is flatter than the classical test pyramid popularised by Cohn (2009), which recommends a large base of unit tests and a comparatively small layer of integration tests. This is explained by the nature of the system under test: a substantial share of its behaviour - role- and policy-based authorization, Entity Framework Core query composition, and cross-module workflows such as a task-status change triggering a notification and an audit-log entry in the same request - can only be meaningfully verified end-to-end through ASP.NET Core's WebApplicationFactory rather than in isolation. This finding is consistent with Sommerville's (2016) observation that integration testing is necessary specifically to catch defects that arise from the interaction of components rather than from any single component in isolation. Testing was not merely confirmatory: it surfaced and led to the correction of concrete defects during development, consistent with the regression-defense role Pressman and Maxim (2020) assign to automated testing in incremental development. Two examples illustrate this. First, an early implementation of the deadline-notification service generated a new overdue-task notification every time the background deadline-check job ran while a task remained overdue, rather than a single notification per occurrence; an integration test that ran the

check twice against the same overdue task exposed the duplication, and the service was corrected to first check for an existing unread notification of the same type before creating a new one. Second, an early implementation of project deletion performed a hard delete that cascaded to a project's tasks, comments and attachments; because this was found, on review against NFR4 (auditability), to destroy exactly the history that an audit trail is meant to preserve, the implementation was revised to a soft-delete pattern using an `IsDeleted` flag and a `DeletedAt` timestamp, paired with a corresponding `AuditLog` entry, and a dedicated integration test was added to confirm that a soft-deleted project's tasks remain retrievable for audit purposes while no longer appearing in normal project listings.

All 106 tests pass at the time of writing (100% pass rate), and the full suite - unit tests against an Entity Framework Core `InMemory` database and integration tests against a dedicated, disposable SQL Server database via `WebApplicationFactory` - is re-run before any further change is merged into the codebase, providing the regression safety net that both Sommerville (2016) and Pressman and Maxim (2020) identify as a precondition for evolving a system incrementally without reintroducing previously-fixed defects.

4.6 Discussion of Findings

Read together, the findings in Sections 4.3 to 4.5 answer the three research questions posed in Section 1.5. On the question of functional and data requirements, Section 4.3 shows that a complete, role-differentiated set of functional and non-functional requirements was derived and that the resulting entity-relationship model was sufficient, without later revision, to support every requirement through to implementation. On the question of how such a system can be designed and developed using Clean Architecture, Section 4.4 shows that the four-layer structure was maintained in practice (not only on paper), that it demonstrably contained the impact of at least one significant change (the addition of Global Search), and that it supported role-scoped presentation without leaking data across roles. On the question of the extent to which the developed system functions correctly, Section 4.5 shows that all 106 automated unit and integration tests pass, and that the testing process itself caught and corrected two non-trivial defects before they could reach a deployed system.

These findings also speak directly to the problem statement in Section 1.2 and the research gap identified in Section 2.3. Where Section 1.2 described ambiguous task ownership, informally tracked deadlines, scattered supporting materials, limited management visibility and weak access control as the consequences of manual coordination, the system's role-scoped task assignment (FR4), automated deadline notifications (FR6), task-attached comments and files (FR5), analytics dashboards (FR7) and role/policy-based authorization (NFR1) directly target each of these consequences, and their correct operation is the specific claim verified in Section 4.5. Where Section 2.3 noted a shortage of academic literature that documents, in reproducible detail, the application of Clean Architecture to a role-based project and task management domain, this chapter's traceability matrix (Table 4.1), module summary (Table 4.2) and test-

result summary (Table 4.3) together constitute exactly that documentation, grounded in the system's own codebase rather than in a purely theoretical account of the architecture.

Overall, the findings support the conclusion that a centralized, role-based, Clean-Architecture web application can satisfy the functional and non-functional requirements of project and task coordination identified in this study, that the architecture's theoretical benefits (testability, change containment, independence from frameworks) are observable in the concrete development history of the system, and that the system's correctness, as far as automated testing can establish it, is well supported by the evidence presented in this chapter. Chapter Five draws on these findings to present the study's conclusions and recommendations.